

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	RENEE VINCENT ASR	Reg. No.:	192424161
Section / Batch:	BTECH AI & DS	Date:	17-08-2026

Code of Conduct

I Renee Vincent ASR-192424161 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Renee Vincent

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.

Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.

Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.

Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

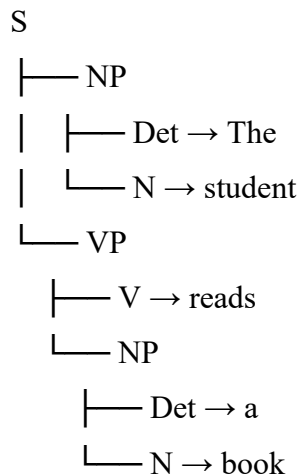
1. CFG Trees vs Dependency Parsing

1. CFG Tree Representation

A **Context-Free Grammar (CFG)** represents a sentence using hierarchical grammatical constituents such as **Sentence (S)**, **Noun Phrase (NP)**, and **Verb Phrase (VP)**.

Example:

"The student reads a book."



CFG mainly shows **how words form larger grammatical phrases**.

Advantages

- Clearly represents phrase structure.
- Useful for grammar checking.
- Helps identify noun phrases and verb phrases.
- Provides a hierarchical representation of sentences.
- Suitable for syntactic parsing and grammar validation.

Limitation

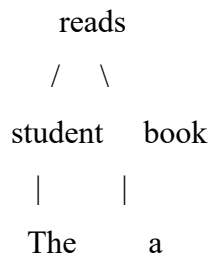
CFG does not directly show the precise relationship between individual words. Extracting relationships such as **who performs an action and what is affected** may require additional analysis.

2. Dependency Parsing Representation

Dependency parsing represents a sentence as relationships between individual words. Each word depends on another word, forming a **dependency tree**.

For:

"The student reads a book."



Relationships include:

student → subject of → reads

book → object of → reads

The → determiner of → student

a → determiner of → book

The main verb **"reads"** acts as the central head.

Advantages

- Directly represents word-to-word relationships.
- Clearly identifies subject, object, and modifiers.
- Useful for semantic role and information extraction.

- Handles long-distance relationships effectively.
- Useful for question answering and information retrieval.

Limitation

- Does not explicitly represent phrase boundaries as clearly as CFG.
- Dependency structures can be more difficult to understand for traditional grammar analysis.
- Different dependency schemes may represent some constructions differently.

3. Comparison

Feature	CFG Trees	Dependency Parsing
Main focus	Phrase structure	Word relationships
Representation	Hierarchical constituents	Head-dependent relationships
Subject identification	Through phrase structure	Direct dependency
Object identification	Through phrase structure	Direct dependency
Phrase boundaries	Strong	Less explicit
Word relationships	Indirect	Direct
Grammar checking	Very useful	Useful
Semantic analysis	Requires additional processing	More suitable
Long-distance relationships	Can be complex	Handles naturally

4. Which Is More Effective?

For **capturing relationships between words, dependency parsing is more effective.**

For example:

"The doctor prescribed medicine to the patient."

Dependency parsing can directly represent:

prescribed

|— doctor → subject
|— medicine → object
└— patient → indirect object

This makes it easy for an NLP system to determine:

Doctor → prescribed → Medicine → to → Patient

A CFG would instead focus on grouping the sentence into noun phrases and verb phrases.

Final Conclusion

CFG trees are better for representing the **hierarchical and phrase-based structure** of a sentence, making them useful for grammar checking and syntactic analysis. **Dependency parsing** focuses directly on **relationships between individual words**, making it more effective for extracting subjects, objects, modifiers, and semantic relationships. Therefore, for applications that require understanding **how words are related**, **dependency parsing is the better choice**, while CFG remains valuable when detailed phrase structure and grammatical constituency are required.

2. Top-Down Parsing vs Earley Parsing

1. Top-Down Parsing

Top-down parsing starts from the **start symbol** of the grammar and tries to generate the input sentence by expanding grammar rules.

Example:

$S \rightarrow NP VP$

$VP \rightarrow V NP$

The parser begins with **S** and works toward the actual input words.

Advantages

- Simple to understand and implement.
- Works efficiently for simple and predictable sentences.
- Requires relatively less memory.
- Suitable when the grammar is small and unambiguous.

Limitations

- May require **backtracking** when an incorrect rule is selected.
- Ambiguous sentences can cause many alternative paths.
- Incomplete input can cause parsing difficulties.
- Processing time may increase for complex sentences.
- Not ideal for highly dynamic conversational input.

2. Earley Parsing

Earley parsing is a **dynamic-programming parsing technique** that can handle general Context-Free Grammars.

It uses three main operations:

Prediction → Scanning → Completion

Instead of repeatedly restarting when an alternative is encountered, Earley parsing stores partial parsing results in a **chart**.

For example, if the user enters:

"Book a flight to..."

Earley parsing can maintain a partial structure showing that the parser is expecting a **destination**.

When the user adds:

"Delhi"

the parser can continue from the existing state.

3. Comparison

Feature	Top-Down Parsing	Earley Parsing
Approach	Start symbol $\rightarrow$ input	Chart-based dynamic parsing
Ambiguity	May require backtracking	Maintains multiple possibilities
Incomplete input	Difficult	Handles partial structures better
Backtracking	Common	Reduced through stored states
General CFG	Depends on implementation	Supports general CFG
Memory usage	Generally lower	Higher due to chart
Simple sentences	Very efficient	May be more processing than necessary
Complex sentences	Can become inefficient	More systematic
Dynamic input	Less suitable	More suitable
Implementation	Simpler	More complex

4. Suitability for Dynamic Input

Consider a real-time voice/chat application receiving:

"Show my account..."

The sentence is incomplete because the user has not finished the request.

A top-down parser may explore possible grammar rules and backtrack when the remaining input does not match the selected structure.

Earley parsing can maintain a **partial parse state** and continue processing when additional words arrive.

For an ambiguous query such as:

"Show transactions with the card..."

Earley parsing can maintain different possible interpretations and resolve them when more context

becomes available.

5. Final Analysis

Top-down parsing is simple and efficient for short, complete, and predictable sentences, but backtracking makes it less effective when input is ambiguous or incomplete. **Earley parsing** stores partial parsing states and can handle ambiguity and incomplete sentences more systematically.

Therefore, **Earley parsing is more suitable for real-time dynamic applications** such as chatbots, voice assistants, and conversational systems where users frequently provide incomplete, long, or ambiguous input. Although it requires more memory and implementation complexity, its flexibility and better handling of dynamic input make it the stronger choice for such applications.

3. CFG vs PCFG vs Neural Parsing

1. Ambiguity in the Sentence

Given sentence:

"She saw the man with a telescope."

The sentence has **prepositional phrase ambiguity** because **"with a telescope"** can have two meanings.

Interpretation 1: Telescope belongs to "She"

She used a telescope to see the man.

She → saw → man

↑

with a telescope

Here, **"with a telescope"** modifies the action **"saw."**

Interpretation 2: Telescope belongs to "man"

She saw a man who had a telescope.

She → saw → man
 ↑
 with a telescope

Here, "**with a telescope**" modifies "**man.**"

Therefore, the parser must determine which interpretation is more likely.

2. CFG Approach

A **Context-Free Grammar (CFG)** uses predefined grammatical rules to generate possible parse trees.

For example:

$S \rightarrow NP VP$

$VP \rightarrow V NP$

$VP \rightarrow VP PP$

$NP \rightarrow NP PP$

CFG can generate **both interpretations** of the sentence.

Advantages

- Simple and rule-based.
- Clearly represents syntactic structure.
- Useful for grammar checking.
- Easy to understand and interpret.

Limitations

- Cannot naturally determine which parse is more likely.
- May produce multiple valid parse trees.

- Requires manually designed grammar rules.
- Has limited understanding of real-world context.

Therefore, CFG can **identify ambiguity but cannot reliably resolve it**.

3. PCFG Approach

A **Probabilistic Context-Free Grammar (PCFG)** extends CFG by assigning probabilities to grammar rules.

For example:

$$P(VP \rightarrow VP PP) = 0.4$$

$$P(NP \rightarrow NP PP) = 0.6$$

The parser calculates the probability of each possible parse tree and selects the one with the highest probability.

Advantages

- Resolves ambiguity using probabilities.
- More flexible than basic CFG.
- Can learn probabilities from a treebank.
- Useful for ranking alternative syntactic structures.

Limitations

- Depends heavily on the quality of training data.
- Basic PCFG does not fully capture semantic context.
- Rule probabilities may not represent deeper language meaning.
- Performance can decrease for unfamiliar sentence structures.

4. Neural Parsing

Neural parsing uses machine-learning models such as **Transformer-based language models** to understand sentence structure using contextual representations.

For:

"She saw the man with a telescope."

A neural parser can consider the surrounding words and learned language patterns to determine whether the telescope is associated with **"saw"** or **"man."**

It can use information from:

- Word context.
- Syntactic patterns.
- Semantic relationships.
- Large-scale training data.
- Long-distance dependencies.

Advantages

- Strong contextual understanding.
- Better handling of complex ambiguity.
- Learns patterns automatically.
- Handles long-distance relationships.
- Performs well on real-world language.
- Requires less manually written grammar.

Limitations

- Requires large training datasets.

- Computationally expensive.
- More difficult to interpret than rule-based systems.
- Can produce incorrect predictions for unfamiliar contexts.
- Requires more computing resources for real-time deployment.

5. Comparison

Feature	CFG	PCFG	Neural Parsing
Ambiguity detection	Good	Good	Excellent
Ambiguity resolution	Poor	Moderate	Strong
Uses probabilities	No	Yes	Yes
Context understanding	Limited	Moderate	Strong
Manual rules	High	Moderate	Low
Training data	Not necessarily required	Required for probabilities	Large datasets usually required
Interpretability	High	High	Lower
Complex sentences	Limited	Better	Strong
Real-world language	Limited	Moderate	Strong
Computational cost	Low	Moderate	High

6. Which Method Is Most Effective?

For **real-world NLP applications**, **neural parsing is generally the most effective** because it can use contextual and semantic information when resolving ambiguity.

For example, consider:

"She saw the man with a telescope."

A neural model can consider the broader context:

"She picked up a telescope before looking at the man."

This strongly suggests that **she used the telescope**.

Alternatively:

"The man was carrying a telescope when she saw him."

This suggests that **the man had the telescope**.

CFG alone cannot make this contextual distinction. PCFG can choose the statistically more likely structure, but neural parsing can incorporate much richer contextual information.

Final Analysis

CFG can represent the two possible syntactic structures but has little ability to choose between them. PCFG improves ambiguity resolution by assigning probabilities to competing parse trees, making it more effective than basic CFG. **Neural parsing** provides the strongest real-world performance because it combines syntactic patterns with contextual and semantic information. Therefore, **neural parsing is the most suitable for modern real-world applications**, while CFG and PCFG remain valuable for interpretable, lightweight, and rule-based systems.

4. Feature Structures vs Subcategorization Frames

1. Feature Structures

Feature structures represent grammatical properties of words and phrases using attributes and values.

For example:

"The student reads a book."

Student:

Number = Singular

Person = Third

Reads:

Number = Singular

Person = Third

Tense = Present

The system checks whether the features of the subject and verb agree.

Subject–Verb Agreement

Correct:

The student reads.

Subject: Singular

Verb: Singular

→ Agreement = Correct

Incorrect:

The student read (*in a present-tense agreement context*)

or:

The students reads.

Subject: Plural

Verb: Singular

→ Agreement = Incorrect

Advantages

- Explicitly represents grammatical features.
- Handles number, person, tense, gender, and case where relevant.
- Enforces subject–verb agreement.
- Helps detect grammatical errors.
- Can represent agreement constraints between different constituents.

2. Subcategorization Frames

Subcategorization frames describe the types and number of arguments that a verb requires.

For example:

eat

eat → Subject + Object

Sentence:

"The student eats food."

The verb **eat** expects an object.

Another example:

sleep

sleep → Subject

Sentence:

"The student sleeps."

The verb does not require a direct object.

For a verb such as **give**:

give → Subject + Object + Recipient

Example:

"The teacher gave the student a book."

- Teacher → Subject
- book → Object
- student → Recipient

3. Main Difference

The key difference is:

Feature structures → Check grammatical properties and agreement.

Subcategorization frames → Check verb argument requirements and relationships.

For example:

"The students eats food."

Feature structures can identify:

students = Plural

eats = Singular

→ Agreement Error

But subcategorization identifies that **eat** requires an appropriate subject and object structure:

eat → Subject + Object

It does not primarily determine whether the subject is singular or plural.

4. Comparison

Feature	Feature Structures	Subcategorization Frames
Main purpose	Grammatical constraints	Verb argument structure
Subject–verb agreement	Excellent	Limited
Number/person	Explicitly represented	Usually not primary
Tense	Can represent	Not primary
Verb arguments	Can represent	Excellent
Required objects	Limited compared with frames	Clearly represented
Grammatical validation	Strong	Moderate
Semantic relationships	Limited	Better for verb arguments
Error detection	Agreement errors	Missing/extra arguments

5. Which Provides Better Grammatical Correctness?

For **enforcing subject–verb agreement and grammatical feature consistency, feature structures provide better support.**

For example:

"She walks to school."

She:

Person = 3

Number = Singular

walks:

Person = 3

Number = Singular

→ Correct

If the sentence is:

"She walk to school."

the feature mismatch can be detected:

Subject = 3rd Person Singular

Verb = 3rd Person Plural/Base

→ Agreement Error

However, feature structures alone are not sufficient for checking whether verbs receive the correct number and type of arguments.

For example:

"The teacher gave."

The sentence may be grammatically incomplete because **give** commonly requires additional arguments in this usage.

A subcategorization frame can detect this:

give $\rightarrow$ Subject + Object + Recipient

Therefore, the two approaches are **complementary rather than competing**.

6. Best Approach

The most effective grammar system should combine both:

Input Sentence



Feature Structure Analysis



Subject–Verb Agreement



Subcategorization Analysis



Verb Argument Validation



Grammatical Decision

For:

"The teacher gave the student a book."

Feature structures verify:

teacher $\rightarrow$ gave

for agreement.

Subcategorization verifies:

gave $\rightarrow$ teacher + student + book

for the required argument structure.

Final Analysis

Feature structures are more effective for enforcing **subject–verb agreement** because they explicitly represent grammatical features such as number, person, and tense. **Subcategorization frames** are more effective for verifying whether a verb receives the correct number and type of arguments. Therefore, for complete grammatical correctness, the best solution is to **combine both approaches**: feature structures enforce agreement constraints, while subcategorization frames validate verb argument structures.

5. Transition-Based vs Graph-Based Dependency Parsing

1. Transition-Based Parsing

Transition-based parsing builds a dependency tree **step-by-step** using a sequence of parsing actions.

Typical operations include:

- **SHIFT** – move a word onto the stack.
- **REDUCE** – remove a processed word.
- **LEFT-ARC** – create a dependency from one word to another.
- **RIGHT-ARC** – create a dependency in the opposite direction.

For example:

"The student reads a book."

The parser gradually creates relationships such as:

student → reads

book → reads

The → student

a → book

Decision-Making

At each step, the parser makes **one local decision** about which transition to perform.

Modern transition-based parsers often use machine-learning models to select the best transition.

Advantages

- Very fast parsing.
- Usually requires relatively low memory.
- Suitable for real-time applications.
- Can process large numbers of sentences efficiently.
- Simple sequential decision process.

Limitations

- Early incorrect decisions can affect later parsing.
- Local decisions may cause error propagation.
- Long-distance dependencies can be more difficult to handle.
- Accuracy depends heavily on the transition classifier.

2. Graph-Based Parsing

Graph-based parsing treats dependency parsing as a **global optimization problem**.

Each possible dependency between words is assigned a score. The parser then searches for the dependency tree with the highest overall score.

For example:

```
      reads
     /  \
student  book
 |      |
The     a
```

The system evaluates possible connections such as:

reads → student

reads → book

student → The

book → a

and selects the combination that produces the best-scoring dependency tree.

Decision-Making

Unlike transition-based parsing, graph-based parsing considers **multiple relationships together** and makes a more global decision.

Advantages

- Considers the entire dependency structure.
- Less affected by individual early parsing errors.
- Often handles complex and long-distance relationships well.
- Provides globally consistent dependency trees.
- Suitable when parsing accuracy is more important than maximum speed.

Limitations

- Generally more computationally expensive.
- Can require more memory.
- Global optimization can be slower on very large datasets.
- Less naturally suited to strict real-time processing.

3. Comparison

Feature	Transition-Based	Graph-Based
Decision type	Local, step-by-step	Global

Processing speed	Very fast	Usually slower
Memory requirement	Lower	Higher
Error propagation	Higher	Lower
Global consistency	Moderate	Strong
Long-distance dependencies	Can be difficult	Better
Large-scale processing	Excellent	Moderate
Real-time applications	Excellent	Less suitable
Implementation	Relatively simpler	More complex
Accuracy	High with good models	Often strong due to global decisions

4. Performance Analysis

For a **large-scale NLP system**, millions of sentences may need to be processed quickly.

Transition-Based Performance

A transition-based parser processes each sentence through a sequence of decisions. Since each decision is relatively inexpensive, it can achieve **high throughput**.

For example:

Input Sentence

↓

SHIFT

↓

SHIFT

↓

RIGHT-ARC

↓

REDUCE

↓

Dependency Tree

This makes it highly suitable for:

- Search engines.
- Social media analysis.
- Chatbots.
- Real-time text processing.
- Large document collections.

Graph-Based Performance

Graph-based parsing evaluates many possible dependency relationships and searches for the best overall tree.

This can provide better global consistency, but the additional computation can become expensive when processing millions of sentences.

Therefore, graph-based parsing is more appropriate when **accuracy and structural consistency are prioritized over maximum throughput.**

5. Which Is More Suitable for Large-Scale Applications?

For the specific requirement of **fast and accurate dependency parsing on big datasets, transition-based parsing is generally more suitable.**

Its main advantages are:

- **High speed**
- **Low computational overhead**
- **Low memory usage**
- **Good scalability**
- **Suitable for real-time processing**

- **Easy integration with machine-learning classifiers**

However, modern systems can combine the strengths of both approaches. For example, a transition-based parser can provide fast initial parsing, while more sophisticated models can be used for difficult or low-confidence sentences.

Final Analysis

Transition-based parsing makes local decisions sequentially and is optimized for speed, making it highly suitable for **large-scale and real-time NLP applications**. **Graph-based parsing** considers the dependency structure globally, which can improve consistency and reduce the effect of local errors, but it generally requires more computational resources. Therefore, for a system processing **large datasets with strong speed requirements, transition-based parsing is the better choice**. For applications where maximum structural accuracy is more important than processing speed, graph-based parsing may be preferred.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____